Group Project

Phase 2

DM857 Introduction to Programming
DS830 Introduction to Programming

A Food Delivery Service – Object-Oriented Extension

This document describes Part II of the annual programming project for the courses DS830 and DM857: Introduktion til Programmering.

In Part I, you implemented a procedural backend for a simplified food-delivery system. The backend was exposed through a fixed functional interface and used by a graphical user interface (GUI) to visualise drivers, requests, and the evolution of the simulation.

In Part II, you will refactor and extend the backend using object-oriented programming (OOP). You will design a small set of interacting classes to model:

- Drivers as active agents (meaning they can change behaviour depending on the situation),
- Customer requests with pickup and drop-off locations, similarly to the previous setup,
- A simulation engine that advances time and keeps global statistics,
- A dispatch policy responsible for proposing which drivers should serve which request. Note: the system is trying to gather highest gain, hence it will offer to multiple drivers the same offer.
- Driver behaviour policies, that decide whether a driver accepts or rejects an offer.
- Mutation rules that allow drivers to change their behaviour over time.

Your new OO backend will still be used through a thin adapter by the same GUI as Part I. This means the high-level observable behaviour (drivers moving on a map, requests appearing and being served) remains familiar, while the internal design and algorithms become significantly more sophisticated.

1 High-Level Design Overview

Conceptually, Part II turns the system into a multi-agent model:

- Requests appear over time and represent food orders.
- A dispatcher proposes assignments between drivers and waiting requests.
- Drivers are active agents that may accept or reject offers according to their current behaviour policy.
- Over time, drivers can mutate their behaviour based on performance (for example earnings or waiting times).

At each simulation step (tick) the following actions occur:

1. New requests appear stochastically, with a configurable rate.
2. Requests that have waited too long may expire.
3. The dispatch policy computes one or more offers: proposals to assign drivers to requests.
4. For each offer, the corresponding driver decides whether to accept or reject, according to its behaviour.
5. The dispatcher resolves conflicts (for example two drivers accept the same request) and finalises assignments.
6. Drivers move according to their current assignment (to pickup or to drop-off).

7. When drivers reach the pickup or drop-off point, the relevant request and driver state is updated, and global statistics are collected.
8. A mutation rule is applied to each driver, which may change its behaviour based on performance.

The rest of this document specifies the required classes and their responsibilities.

2 Core Domain Classes

2.1 Class `Point`

A simple value class representing a position on the map.

- Attributes:
  - `x: float`
  - `y: float`
- Methods:
  - `distance_to(other: Point) -> float`
    Returns the Euclidean distance between this point and`other`.
  - Define the special methods:
    `__add__`, `__sub__`, `__iadd__`, `__isub__`, `__mul__` and `__rmul__`
    where the last two are meant to be defined both for multiplcations for `int` and `float`

This class is intended to be used throughout the positions' evolution.

2.2 Class `Request`

Represents a single food-delivery request with pickup and drop-off locations.

- Attributes (minimum):
  - `id: int`
  - `pickup: Point`
  - `dropoff: Point`
  - `creation_time: int` (simulation tick at which the request appeared)
  - `status: str` (for example`"WAITING"`,`"ASSIGNED"`,`"PICKED"`,`"DELIVERED"`,`"EXPIRED"`)
  - `assigned_driver_id: int | None`
  - `wait_time: int` (time spent in the system until pickup/delivery)
- Methods (suggested):
  - `is_active() -> bool`
    Returns `True` if the request is still waiting, assigned, or picked (that is, not delivered or expired).
  - `mark_assigned(driver_id: int) -> None`
  - `mark_picked(t: int) -> None`
  - `mark_delivered(t: int) -> None`
  - `mark_expired(t: int) -> None`
  - `update_wait(current_time: int) -> None`
    Updates `wait_time` according to `current_time`.

2.3 Class `Driver`

Represents a driver agent that can move on the map and accept or reject offers.

- Attributes (minimum):
  - `id: int`
  - `position: Point`

  - `speed: float` (units per tick)
  - `status: str` (for example`"IDLE"`,`"TO_PICKUP"`,`"TO_DROPOFF"`)
  - `current_request: Request | None`
  - `behaviour: DriverBehaviour`
  - `history: list` (to store recent completed trips, earnings, and similar statistics)
- Methods (minimum):
  - `assign_request(request: Request, current_time: int) -> None`
  - `target_point() -> Point | None`
    Returns the next target: the pickup point if status is`"TO_PICKUP"`, or the drop-off point if status is`"TO_DROPOFF"`.
  - `step(dt: float) -> None`
    Moves the driver towards the current target according to speed and the time step `dt`.
  - `complete_pickup(time: int) -> None`
    Updates internal state when the pickup is reached.
  - `complete_dropoff(time: int) -> None`
    Updates internal state and history when the drop-off is reached.

3 Policies, Behaviour, and Mutation

3.1 Class `DispatchPolicy`

An **abstract** base class representing the central dispatch strategy.

- Responsibility: given the current state (drivers and active requests), propose assignments of drivers to requests.
- Interface:

```
class DispatchPolicy:
    def assign(self,
               drivers: list[Driver],
               requests: list[Request],
               time: int
              ) -> list[tuple[Driver, Request]]:
        """Return proposed (driver, request) pairs for this tick."""
        raise NotImplementedError
```

You must implement at least the following polycies::

- **NearestNeighborPolicy**: repeatedly match the closest idle driver to the closest waiting request.
- **GlobalGreedyPolicy**: build all (idle driver, waiting request) pairs, sort by distance, and match greedily while avoiding reuse of drivers and requests.

Optionally, you may implement a more advanced policy (for example to minimise total waiting time or travel distance using more sophisticated algorithms).

3.2 Class `Offer`

A simple data object describing a proposal from the dispatcher to a driver.

- Attributes (suggested):
  - `driver: Driver`
  - `request: Request`
  - `estimated_travel_time: float`
  - `estimated_reward: float` (if you choose a reward model)

3.3 Class `DriverBehaviour`

An abstract base class encoding how a driver decides whether to accept an offer.

- Interface:

```
class DriverBehaviour:
    def decide(self,
               driver: Driver,
               offer: Offer,
               time: int
              ) -> bool:
        """Return True if the driver accepts the offer, False otherwise."""
        raise NotImplementedError
```

Concrete behaviours should include, for example:

- **GreedyDistanceBehaviour**: accept if the distance to the pickup is below a given threshold.
- **EarningsMaxBehaviour**: accept if the ratio estimated reward divided by travel time is above a threshold.
- **LazyBehaviour**: accept only if the request is close and the driver has been idle for longer than a configurable number of ticks.

You must implement at least two distinct behaviours and ensure that `Driver` can be constructed with any behaviour and can change behaviour at runtime.

3.4 Class `MutationRule`

Models how drivers change their behaviour over time.

- Responsibility: inspect a driver (and possibly global statistics) and decide whether to update its behaviour or behaviour parameters.
- Interface:

```
class MutationRule:
    def maybe_mutate(self, driver: Driver, time: int) -> None:
        """Possibly change the driver's behaviour based on performance."""
        raise NotImplementedError
```

Example mutation rules:

- Performance-based mutation: if the driver's average earnings or served requests in the last N trips is below a threshold, change from a picky to a more greedy behaviour.
- Exploration: with a small fixed probability, randomly switch to another behaviour.

You must implement at least one non-trivial mutation rule and demonstrate that drivers can change behaviour during a simulation run.

4 Request Generation

4.1 Class `RequestGenerator`

The arrival of new requests should follow a fixed average rate (as in Part I), but now modelled as a separate object.

- Attributes (suggested):
  - `rate: float` (expected number of new requests per tick),
  - a random number generator,
  - `next_id: int` (for request identifiers),

  - map boundaries (width and height).
- Methods:

```
class RequestGenerator:
    def maybe_generate(self, time: int) -> list[Request]:
        """
        Called once per tick. Draws, according to a user's defined rule, and returns N new R
        objects whose creation_time is 'time' and whose pickup/dropoff points
        are valid positions in the map.
        """
```

5 Simulation Engine and Adapter Contract

5.1 Class `DeliverySimulation`

This class orchestrates the entire simulation.

- Attributes:
  - `time: int`
  - `drivers: list[Driver]`
  - `requests: list[Request]` (including active and completed)
  - `dispatch_policy: DispatchPolicy`
  - `request_generator: RequestGenerator`
  - `mutation_rule: MutationRule`
  - `timeout: int` (maximum allowed waiting time before expiration)
  - statistics such `asserved_count`,`expired_count`,`avg_wait`.
- Key methods:

```
class DeliverySimulation:
    def tick(self) -> None:
        """Advance the simulation by one time step."""
        # 1. Generate new requests.
        # 2. Update waiting times and mark expired requests.
        # 3. Compute proposed assignments via dispatch_policy.
        # 4. Convert proposals to offers, ask driver behaviours to accept/reject.
        # 5. Resolve conflicts and finalise assignments.
        # 6. Move drivers and handle pickup/dropoff events.
        # 7. Apply mutation_rule to each driver.
        # 8. Increment time.

    def get_snapshot(self) -> dict:
        """
        Return a dictionary containing:
          - list of driver positions and headings,
          - list of pickup positions (for WAITING/ASSIGNED requests),
          - list of dropoff positions (for PICKED requests),
          - statistics (served, expired, average waiting time).
        Used by the GUI adapter.
        """
```

5.2 Adapter Functions

As in Part I, the GUI will interact with your backend through a small set of functions. At a minimum, you must provide functions (in a separate module) that create and manipulate a `DeliverySimulation` instance:

- `init_simulation(...params...) -> None`,
- `step_simulation() -> tuple[int, dict]` (returns current time and metrics),
- `get_plot_data() -> ...` (returns positions needed for plotting).

The exact function signatures will be given in the project repository, and will mirror the Part I interface as closely as possible. Your task is to construct and manage the `DeliverySimulation` object behind these functions.

6 Class Diagrams

This section provides schematic diagrams to illustrate the intended relationships among classes. You are free to extend the design, provided you respect the core ideas and the adapter contract.

Core Structure

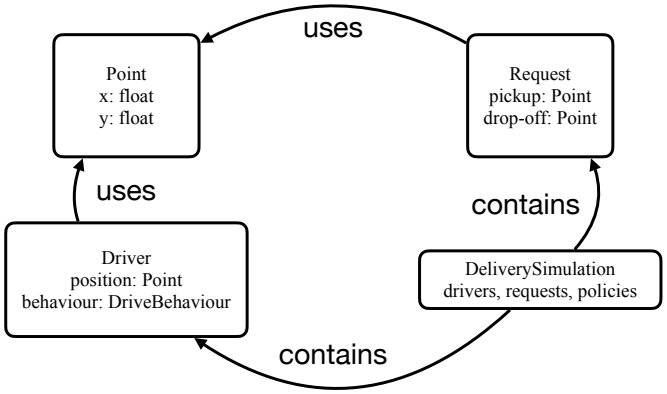

Figure 1: Core classes and containment relations.

Policies, Behaviour, and Mutation

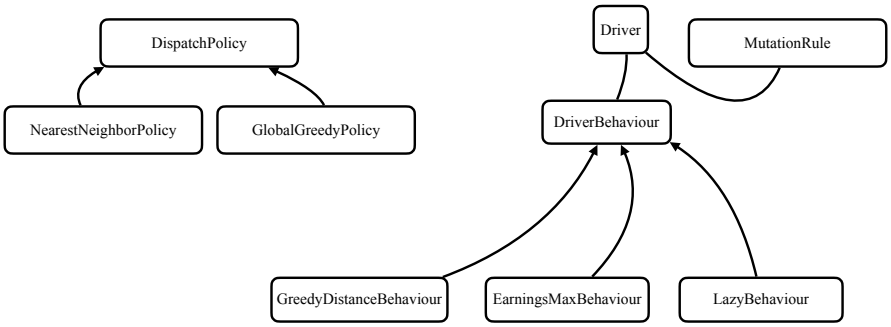

Figure 2: Policy, behaviour, and mutation classes. Arrows from concrete policies and behaviours to the abstract base denote inheritance.

7 Post-Simulation Metrics and Reporting

After closing the graphical application window, the program should display a new window or generate plots summarizing the evolution of the simulation metrics over time. This secondary output is intended to allow inspection and analysis of how the system performed during the simulation — for example, trends in served or expired requests, or the progression of average waiting times — and should make it possible to distinguish the earnings achieved under different policies.

8 Hand-in

8.1 Files

For this phase, submit a zip file containing:

- A folder named `code` with your implementation, including a module named `phase2`.
- A PDF document titled `named_report.pdf` containing your report. The zip file should be named after your group, e.g., `group_B40.zip`.

8.2 Setup

- Your solution must contain a module named `phase2`, which serves as the main module of your program; otherwise, you may organize your solution as you see fit.
- Do not modify any modules provided with this assignment (`_engine.py`).
- Document your code, adhering to the Python coding conventions and course rules.
- Use only modules from the Python standard library, plus `numpy`, `matplotlib`, `os`,and `dearpygui` (other third-party packages are not allowed).
- A functional unit-testing implementation is required.

- In your report, describe your implementation, explaining and justifying its structure and functionality.
- You should analyse the scaling behaviour of your code and discuss the expected performance characteristics.
- Include your code[1] as an appendix. To save space, you may summarize or omit docstrings, doctests, and code for printing statistics.
- No specific report structure is required beyond including a code appendix.
- The report must be in English and delivered as a single PDF, printable in black and white (avoid dark backgrounds for code listings) and limited to 10 pages, excluding the appendix.
- The report must state the group's name and list its members (full names and SDU emails, if applicable).

[1] If preparing your report in LaTeX, use packages such as "minted" or "listings" for source code.